Como o frontend funciona

FATECalçados · Programa de fidelidade · telas, chamadas à API, acessibilidade e bibliotecas · setembro de 2026 · pasta `frontend/src/`

Este documento descreve o frontend do projeto: como cada tela chama a API do backend, como os dados circulam entre as telas, quais recursos de acessibilidade existem e quais bibliotecas são usadas. Os detalhes de cada endpoint no servidor (campos, respostas, erros) estão em `docs/endpoints-da-api.pdf`; aqui o foco é o lado do navegador.

1. Visão geral

O frontend é uma aplicação React 19 em TypeScript (modo `strict`), empacotada pelo Vite e estilizada com Tailwind CSS v4. Ele tem duas áreas com públicos diferentes, e uma parte pública:

Área	Endereços	Quem usa	Layout
Pública	`/`, `/login`, `/cadastro`, `/privacidade`, `/admin/entrar`	Qualquer visitante	Próprio (landing e `TelaAcesso`)
Cliente	`/loja`, `/loja/produto/:sku`, `/loja/carrinho`, `/minha-conta`, `/minha-conta/compras`, `/minha-conta/avaliacoes`	Cliente logado (CPF + senha)	`ClienteLayout`
Loja	`/admin/pdv`, `/admin/dashboard`, `/admin/clientes`, `/admin/vendas`, `/admin/acoes`, `/admin/avaliacoes`, `/admin/estoque`	Equipe da loja (usuário `is_staff`)	`MainLayout`

As rotas ficam em `routes/index.tsx` (`createBrowserRouter` do React Router). A área da loja só é alcançada digitando `/admin`; a landing leva apenas a `/login` e `/cadastro`. Endereços desconhecidos dentro de `/admin` mostram a tela `NotFound` dentro do layout; qualquer outro volta para `/`.

Organização das pastas

Pasta	Papel
`pages/<área>/`	Uma pasta por tela (pdv, dashboard, clientes, vendas, acoes, avaliacoes, estoque, conta, inicio). As páginas só montam componentes.
`hooks/`	Lógica das telas: carrinho, fila offline, atalhos, carregamento, tema, acessibilidade, tour.
`contexts/`	Estado que precisa sobreviver à troca de tela (caixa, ações, conta do cliente, sessão da equipe, avisos).
`services/api/`	Único lugar que chama a API. Um módulo por área, cada um convertendo os nomes em português da API para os nomes usados nas telas.
`components/ui/`, `components/layout/`	Peças compartilhadas (Button, Card, Modal, Abas, Estrelas…) e o esqueleto (Sidebar, busca, painéis de ajuda e acessibilidade, tour).
`regras.ts`	Cópia das regras de pontos de `regras.py` (100 pontos = R$ 5,00, níveis, validade) usada só para prévias e explicações. O backend é quem decide.
`utils/`	Funções puras usadas em mais de um lugar: formatação de dinheiro e datas, CPF, operações do carrinho.

2. Como uma chamada à API acontece

```
Tela (ex.: Vendas.tsx)
  └─ fetchSales({ start, end, page })            services/api/vendas.ts
       └─ request("/api/vendas/?inicio=…&pagina=2")   services/api/http.ts
            └─ fetch → proxy do Vite → Django (backend:8000)
       ←─ JSON em português  { valor_pago, forma_pagamento, … }
  ←─ objeto em inglês        { total, payment, … }   (mapeador toSale…)
```

O arquivo `http.ts`

- `request<T>(url, {method, body})`: faz o `fetch` com `Content-Type: application/json` e o cabeçalho `X-CSRFToken`, lido do cookie `csrftoken`. Se a rede falhar, lança `ApiError("Sem conexão com o servidor.", 0)`; se a resposta não for 2xx, lança `ApiError` com a mensagem do campo `erro` que o backend mandou e o código HTTP (`status`).
- `ensureCsrfCookie()`: em `/login`, `/cadastro` e `/admin/entrar` ainda não houve nenhum GET, então o cookie CSRF pode não existir. Antes do POST de login, faz um GET em `/api/conta/` só para receber o cookie (o 401 da resposta não importa).
- `errorMessage(e)`: transforma qualquer erro capturado em texto para mostrar na tela.
- `queryString(params)`: monta `?a=1&b=2` ignorando parâmetros vazios.

A sessão (do cliente e da equipe) é o cookie `sessionid` do Django, enviado pelo navegador automaticamente, porque o frontend e a API ficam na mesma origem graças ao proxy. Por isso nenhuma rota é `csrf_exempt`: todo POST leva o token CSRF.

Proxy: `vite.config.ts` repassa tudo que começa com `/api` para `API_URL` (`http://backend:8000` no Docker, `http://127.0.0.1:8000` fora dele; no Windows `localhost` vira IPv6 e trava no Docker Desktop). Por isso nenhuma rota do React pode começar com `/api`.

O hook `useCarregar`

As telas de lista (Dashboard, Clientes, Vendas, Avaliações, Estoque) carregam dados por `useCarregar(load, key)`. A `key` é o conjunto de filtros serializado: quando muda, a função é chamada de novo e a resposta antiga, se chegar depois, é descartada. Os dados anteriores continuam na tela durante a recarga, e o hook devolve `{ data, error, loading, retry }` para a tela mostrar o estado de carregamento, a mensagem de erro e o botão "Tentar de novo" (`ErrorState`).

3. Funções da API e onde são usadas

Todas saem de `services/api/index.ts`, que exporta só as funções de busca (os mapeadores ficam de fora). Acesso: **Aberto** = sem login; **Equipe** = sessão de usuário da loja (senão 401); **Cliente** = sessão do cliente.

Função	Endpoint	Acesso	Onde é chamada	O que faz
Produtos (`produtos.ts`)				
`fetchProducts`	GET `/api/produtos/`	Aberto	`CaixaProvider`, `ContaProvider`	Catálogo com preço, estoque, grade de numerações e média de avaliações. Carregado uma vez por área e compartilhado.
`fetchProductDetail`	GET `/api/produtos/<sku>/`	Aberto	`conta/Produto.tsx`	Página do produto: descrição, estoque por numeração, avaliações e "Você também pode gostar".
Equipe (`equipe.ts`)				
`fetchStaff`	GET `/api/equipe/`	Equipe	`EquipeProvider`	Confere a sessão antes de abrir `/admin`; 401 vira `null` e leva para `/admin/entrar`.
`staffSignIn`	POST `/api/equipe/entrar/`	Aberto	`inicio/EntrarEquipe.tsx`	Login com usuário e senha; volta para a página pedida (`state.de`).
`staffSignOut`	POST `/api/equipe/sair/`	Equipe	Botão "Sair" da sidebar	Encerra a sessão.
PDV / Clientes (`clientes.ts`, `vendas.ts`)				
`fetchClient`	GET `/api/clientes/<cpf>/`	Equipe	`pdv/ClientPanel.tsx`	Identifica o cliente pelo CPF no caixa. 404 vira `null` ("CPF sem cadastro").
`registerClient`	POST `/api/clientes/`	Equipe	`pdv/ClientPanel.tsx`	Cadastro rápido (CPF + nome + código do caixa); o cliente já ganha pontos na compra atual.
`sendSale`	POST `/api/vendas/`	Equipe	`services/filaVendas.ts`	Envia a venda (`id` UUID do caixa, `caixa`, `cpf`, itens com `sku` / `quantidade` / `numeracao`, pagamento, pontos a resgatar, `data`). Recebe o valor pago, desconto e pontos oficiais.
`fetchClients`	GET `/api/clientes/?busca=&nivel=&pagina=&ordem=`	Equipe	`clientes/Clientes.tsx`, `SearchPalette`	Lista paginada de clientes (CPF mascarado), ordenada por saldo ou por nome.
`fetchClientProfile`	GET `/api/clientes/<id>/perfil/`	Equipe	`clientes/ClientePerfil.tsx`	Perfil: nível, progresso, saldo, lotes a vencer, extrato e compras.
`fetchSales`	GET `/api/vendas/?inicio=&fim=&caixa=&pagamento=&pagina=`	Equipe	`vendas/Vendas.tsx`	Lista de vendas filtrada e paginada.
`fetchSaleDetail`	GET `/api/vendas/<id>/`	Equipe	`vendas/SaleDetailModal.tsx`	Itens, descontos e pontos de uma venda.
Gestão (`dashboard.ts`, `acoes.ts`, `avaliacoes.ts`, `estoque.ts`)				
`fetchDashboard`	GET `/api/dashboard/?inicio=&fim=`	Equipe	`hooks/useDashboard.ts`	Indicadores do período: pontos emitidos/resgatados/vencidos, passivo, vendas, níveis, ranking, produtos.
`fetchActions`	GET `/api/acoes/`	Equipe	`AcoesProvider`	Listas de ação do gerente (pontos vencendo, perto de subir de nível, inativos...). Alimenta a tela e o contador da sidebar.
`fetchStoreReviews`	GET `/api/avaliacoes/?produto=&nota=&pagina=`	Equipe	`avaliacoes/AvaliacoesLoja.tsx`	Resumo, distribuição de notas, média por produto e comentários paginados.

`fetchStock`	GET `/api/estoque/?situacao=`	Equipe	`estoque/Estoque.tsx`	Estoque por produto e numeração, com situação (OK / Acabando / Esgotado) e vendas dos últimos 30 dias.
`updateStock`	POST `/api/estoque/<sku>/`	Equipe	`estoque/Estoque.tsx`	Contagem de uma numeração (ou do acessório): quantidade e mínimo. Devolve a linha atualizada, que substitui a da lista.
Área do cliente (`conta.ts`)				
`fetchAccount`	GET `/api/conta/`	Cliente	`ContaProvider`	Painel do cliente (saldo, nível, lotes, compras, pendências de avaliação). 401 vira `null` e leva para `/login` .
`signIn`	POST `/api/conta/entrar/`	Aberto	`inicio/Login.tsx`	Login por CPF + senha. Já devolve o painel, que vai no `state` da navegação para a loja abrir sem esperar.
`createAccess`	POST `/api/conta/cadastro/`	Aberto	`inicio/Cadastro.tsx`	Cria o acesso (CPF, nome, senha); liga a um cliente do caixa se o primeiro nome bater.
`signOut`	POST `/api/conta/sair/`	Cliente	`ContaProvider`	Encerra a sessão do cliente.
`updateProfile`	POST `/api/conta/perfil/`	Cliente	`conta/MeusDados.tsx`	Salva numeração, nascimento e autorização de contato.
`deleteAccount`	POST `/api/conta/excluir/`	Cliente	`conta/MeusDados.tsx`	Exclusão (LGPD) com confirmação de senha; o backend anonimiza a conta.
`buyOnline`	POST `/api/conta/compras/`	Cliente	`conta/Checkout.tsx`	Finaliza a compra online (`id` UUID, itens, pagamento, pontos). O CPF vem da sessão, nunca do corpo.
`fetchAccountPurchase`	GET `/api/conta/compras/<id>/`	Cliente	`SaleDetailModal` (`doCliente`)	Detalhe de uma compra do próprio cliente.
`fetchReviews`	GET `/api/conta/avaliacoes/`	Cliente	`conta/Avaliacoes.tsx`	Produtos a avaliar e avaliações feitas, com os números da regra (pontos, tamanho mínimo do comentário).
`sendReview`	POST `/api/conta/avaliacoes/`	Cliente	`conta/Avaliacoes.tsx`	Envia nota (1–5) e comentário; o cliente ganha 20 pontos.

4. Funcionamento de cada área

4.1 Entrada da equipe e proteção de `/admin`

`MainLayout` é envolvido pelo `EquipeProvider`, que chama `fetchStaff` **antes** de montar o caixa e as ações. Sem sessão, redireciona para `/admin/entrar` guardando a página pedida; depois do login, a pessoa volta para ela. Como cliente e equipe dividem a mesma sessão do Django, entrar como um no navegador desconecta o outro.

4.2 PDV / Caixa (`/admin/pdv`)

- **Produtos:** vêm do `CaixaProvider` (carregados uma vez). A busca (`ProductSearch`) e a grade (`ProductGrid`) filtram localmente. Clicar num tênis abre `SeletorNumeracao` com os pares por numeração, coloridos pela situação do estoque; uma numeração zerada continua clicável, porque o caixa nunca recusa venda (o par já está na mão do cliente).
- **Cliente:** `ClientPanel` valida o CPF (`utils/cpf.ts`, mesma regra do `cpf.py`), chama `fetchClient` e, se não houver cadastro, oferece `registerClient`.
- **Pontos e totais:** `useCarrinho` calcula subtotal, desconto do resgate e a prévia dos pontos com `regras.ts`. É só prévia: o valor oficial vem na resposta de `sendSale`.
- **Atalhos:** F2 foca a busca de produtos, F10 finaliza a venda (`useAtalhosPdv`).
- **Venda offline:** cada venda recebe um `id` `crypto.randomUUID()` no caixa. `useFilaVendas` grava a fila em `localStorage` (`fatec:vendas`). Online, a venda é enviada na hora; offline (botão "Simular queda de internet"), fica pendente. Ao reconectar ou recarregar a página, `sendPendingSales` reenvia uma a uma; as que falham continuam pendentes. Como o backend é idempotente pelo `id`, reenviar nunca duplica. Um 401 durante o reenvio deixa as vendas pendentes.
- **Venda em andamento** (carrinho, cliente, pagamento) fica no `CaixaProvider`, então não se perde ao trocar de aba; é limpa assim que a venda é registrada.

4.3 Dashboard (`/admin/dashboard`)

`useDashboard` chama `fetchDashboard` com o período da URL (`?inicio=&fim=`, via `usePeriodoNaUrl`), então um link copiado abre o mesmo recorte. A página é dividida em seções (Pontos, Vendas, Clientes) com cores de significado fixo (`tons.ts`: azul = pontos ganhos, verde = resgatados, âmbar = custo/a vencer, vermelho = vencidos, violeta = base de clientes, ciano = vendas). "Exportar CSV" monta o arquivo no navegador (`csv.ts`); "Exportar PDF" é `window.print()` com um layout de impressão em `@media print`, forçando o tema claro durante a impressão (`useTemaClaroAoImprimir`).

4.4 Clientes e Vendas

Filtros, ordenação e página ficam na URL, então voltar no navegador restaura a lista. Clientes abre o perfil em `/admin/clientes/:id` (`fetchClientProfile`); Vendas abre o detalhe num modal (`fetchSaleDetail`). O CPF chega sempre mascarado. A busca global (Ctrl/⌘ K, `SearchPalette`) procura telas e clientes com `fetchClients`; ali e na lista de Clientes a busca espera a pessoa parar de digitar (`useDebounce`).

4.5 Ações, Avaliações e Estoque

- **Ações:** `AcoesProvider` guarda as listas e quais foram marcadas como feitas; o contador na sidebar vem daí.
- **Avaliações da loja:** resumo e média por produto cobrem todas as avaliações; os filtros `produto` / `nota` (na URL) só estreitam a lista de comentários.
- **Estoque:** cada tênis mostra a grade de numerações colorida (`situacaoDoEstoque.ts`); clicar numa numeração abre a edição, que chama `updateStock` e troca só aquela linha.

4.6 Área do cliente

- **Sessão:** `ClienteLayout` usa o `ContaProvider`. Se o login acabou de devolver o painel, ele aparece na hora e é atualizado em segundo plano com `fetchAccount`; sem sessão, volta para `/login`.
- **Loja e produto:** `Loja` reaproveita a grade do PDV com cartões de vitrine (`CardProduto`: foto, média, "+N pts", "Tem o seu nº 40", selos de esgotado/últimas unidades). A página do produto pré-seleciona a numeração do perfil, risca as esgotadas e limita a quantidade ao estoque menos o que já está no carrinho.
- **Carrinho e checkout:** o carrinho online (`{sku, size, quantity}`) fica em `fatec:carrinho-online`. `useCarrinhoOnline` soma totais e o estoque disponível de cada linha, travando o "+" e marcando itens que esgotaram. O checkout gera o `id` da compra uma vez, então um duplo clique ou reenvio não cria duas compras. Diferente do caixa, a loja online recusa compra sem estoque (o backend responde 400 e a mensagem aparece).
- **Minha conta:** abas Pontos, Extrato e Meus dados, com a aba no hash da URL (`#pontos`, `#extrato`, `#dados`). Em Meus dados, edições não salvas ficam em `sessionStorage` para sobreviver à ida a `/privacidade`; o cartão "Excluir minha conta" pede a senha num modal.
- **Notificações:** o sino da sidebar lista avisos calculados do painel por `notificacoes.ts` (aniversário, pontos vencendo, produtos para avaliar, perto do próximo nível, perfil incompleto). Nada fica no banco; as lidas ficam em `fatec:notificacoes-lidas:<id>`.

4.7 O que fica guardado no navegador

Chave	Onde	Conteúdo

`fatec:vendas`	localStorage	Fila de vendas do caixa (pendentes e as de hoje)
`fatec:carrinho-online`	localStorage	Carrinho da loja online
`fatec:tema`, `fatec:acessibilidade`	localStorage	Tema e opções de acessibilidade
`fatec:sidebar`, `fatec:tour`	localStorage	Sidebar recolhida; tour já visto
`fatec:notificacoes-lidas:<id>`	localStorage	Notificações lidas por cliente
`fatec:rascunho-perfil:<id>`	sessionStorage	Rascunho do formulário Meus dados

Toda leitura e escrita é protegida por `try/catch`: sem armazenamento (aba anônima, bloqueio), o sistema continua funcionando só em memória. Os únicos cookies são `sessionid` e `csrftoken`, ambos essenciais, por isso não há banner de consentimento.

5. Acessibilidade

5.1 Painel "Aparência e acessibilidade"

Uma entrada única na sidebar (`PainelAcessibilidade.tsx`) reúne:

Opção	Efeito	Como funciona
Tema	Sistema, Claro ou Escuro	`data-theme` no `<html>`; as cores são variáveis CSS, então todo o site troca junto
Tamanho da letra	Normal, Grande, Muito grande	`data-fonte` aumenta a fonte base
Alto contraste	Textos e bordas mais escuros, contorno de foco mais grosso	`data-contraste="alto"`, com versões para claro e escuro
Espaçamento de texto	Mais espaço entre linhas, letras e palavras	`data-espacamento="amplo"`
Reduzir animações	Desliga transições e efeitos	`data-movimento="reduzido"`; o CSS também respeita `prefers-reduced-motion` do sistema
Sublinhar links	Links sempre sublinhados	`data-links="sublinhados"`

As escolhas valem na hora e ficam salvas. Um script no `index.html` aplica tudo antes da primeira pintura, para a página não abrir com a letra pequena ou no tema errado e depois pular. O painel usa controles nativos (rádios e checkboxes com `<label>`, agrupados em `<fieldset>` / `<legend>`; os interruptores são checkboxes com `role="switch"`), que leitores de tela e teclado já entendem.

5.2 Teclado e foco

- **"Pular para o conteúdo":** primeiro link dos dois layouts, leva ao `<main id="conteudo">`.
- **Modais** (`Modal.tsx`): `role="dialog"` + `aria-modal`; o foco entra no modal, o Tab fica preso dentro dele, Esc fecha, e ao fechar o foco volta ao botão que o abriu.
- **Abas** (`Abas.tsx`): padrão WAI-ARIA (`tablist` / `tab` / `tabpanel`, `aria-selected`, `aria-controls`); só a aba ativa entra no Tab, e as setas, Home e End trocam de aba.
- **Buscas com lista** (busca global e busca de produtos do PDV): `role="combobox"` com `aria-activedescendant`, navegação por setas e Enter.
- **Atalhos:** Ctrl/⌘ K (busca), F2 e F10 no PDV, mostrados na tela com `<kbd>`.
- Foco visível em todos os controles (`focus-visible`), mais grosso no alto contraste.

5.3 Leitores de tela

- Estados de carregamento, avisos e resultados usam `role="status"` / `aria-live` (ex.: "Procurando cliente...", a quantidade do item no carrinho após − ou +), e erros usam `role="alert"`.
- Ícones decorativos (lucide) levam `aria-hidden="true"`; botões só com ícone têm `aria-label` ("Fechar", "Remover"...).
- Barras de progresso de nível são `role="progressbar"` com `aria-valuenow/min/max`; colunas ordenáveis da lista de clientes informam `aria-sort`.
- As estrelas de avaliação (`Estrelas.tsx`) são rádios nativos, então a nota é escolhida e anunciada como um grupo de opções.
- Textos só para leitor de tela usam a classe `sr-only`; o documento declara `lang="pt-BR"`, e a cada troca de tela o título da aba muda e o foco vai para o título da página, que o leitor de tela anuncia (`useTituloDaRota`); filtros na URL não tiram o foco do controle.
- Cor nunca é a única informação: a situação do estoque e os níveis também aparecem em texto (legenda, "Esgotado", "Acabando", "Prata").

5.4 Telas pequenas e impressão

O layout foi conferido até 320px de largura: a sidebar vira gaveta, tabelas escondem colunas secundárias abaixo de 640px (`colunaSecundaria`) mantendo a coluna principal visível, e grades de produto passam a 2 colunas abaixo de 520px. Na impressão, sidebar, cabeçalho e filtros somem e o tema claro é forçado.

5.5 Tour e ajuda

Na primeira visita à área da loja, o `TourGuiado` destaca cada item da sidebar (abrindo a gaveta no celular) e pode ser pulado ou revisto pelo botão "Rever o tour". "Como funcionam os pontos" (`PainelAjuda`) explica as regras em linguagem do cliente ou da loja, com os números de `regras.ts`.

6. Bibliotecas

Dependências da aplicação (vão para o navegador)

Biblioteca	Versão	Uso no projeto
`react`, `react-dom`	19.2	Base da interface: componentes, estado, contextos e hooks.
`react-router-dom`	7.18	Rotas (`createBrowserRouter`), layouts aninhados com `<Outlet />`, redirecionamentos, parâmetros e filtros na URL, estado passado na navegação.
`recharts`	3.10	Gráfico de barras de pontos do Dashboard (`components/charts/PointsChart.tsx`). As outras barras (vendas por pagamento e por caixa) são HTML/CSS simples.
`lucide-react`	1.48	Ícones em SVG (sidebar, botões, estados), importados um a um para o pacote ficar pequeno.

Ferramentas de desenvolvimento

Biblioteca	Versão	Uso no projeto
`vite` + `@vitejs/plugin-react`	8.3 / 6.1	Servidor de desenvolvimento com recarga instantânea, proxy de `/api` e build de produção.
`typescript`	6.0	Tipagem estrita; `tsc -b` roda antes do build. Os tipos `Api...` descrevem o JSON do backend.
`tailwindcss` + `@tailwindcss/vite`	4.3	Estilos por classes utilitárias; cores do tema como variáveis CSS mapeadas com `@theme inline`, trocando para o escuro sozinhas.
`vitest`	5.0	Testes (`npm test`): regras, utilitários, CSV e toda a camada `services/api` contra um `fetch` falso com respostas de exemplo tipadas.
`oxlint`	1.81	Análise estática (`npm run lint`), incluindo a regra de só exportar componentes nos arquivos `.tsx`.
`@types/react`, `@types/react-dom`, `@types/node`	—	Tipos para o TypeScript.

Recursos externos e APIs do navegador

- **Fonte Geist**, carregada do Google Fonts.
- **Fotos** dos produtos e da landing: Unsplash (licença livre) e uma do Pixabay (Palmilha Gel).
- `InkReveal` (fundo das telas de login): componente adaptado do 21st.dev, desenhado em `<canvas>`, sem biblioteca extra.
- APIs nativas: `fetch`, `localStorage` / `sessionStorage`, `crypto.randomUUID()` (ids de venda e compra), `toLocaleString("pt-BR")` (moeda e números), `window.print()`.

O projeto não usa biblioteca de estado global (Redux, Zustand), de requisições (Axios, React Query) nem de componentes prontos: o estado compartilhado fica nos contextos do React, as requisições no `http.ts`, e os componentes em `components/ui/`.

7. Qualidade

Comando (em `frontend/`)	O que verifica
`npm run lint`	oxlint
`npm run build`	Tipos (`tsc -b`) e build do Vite
`npm test`	Vitest: `regras.test.ts`, `utils/*.test.ts`, `csv.test.ts`, `notificacoes.test.ts` e `testes/api.test.ts` (cada função da API com respostas simuladas em `testes/dados/`)

Quando um campo da API muda, o tipo `Api...`, a resposta de exemplo e o teste são atualizados juntos; o `tsc` acusa a resposta de exemplo desatualizada. Não há testes de componentes.